Report of 1st Deliverable of Project

CSI 2132

Haojian Wang, 300411829
Xinran Xie, 300452622
25 Feb 2026

a. Attributes & ER Diagram

Attributes
1.1 **Hotel Chain**

```
HotelChain(chain_id, central_office_address, category, num_hotels, {email}, {phone})
```

1.2 **Hotel info**

```
Hotel(hotel_id, address, num_rooms, {email}, {phone})
```

1.3 **Rooms**

```
Room(room_id, price, capacity, view_type, extendable, has_problem, {amenity})
```

1.4 **Customers**

```
Customer(customer_id, full_name, address, id_type, id_value, registration_date)
```

1.5 **Employees**

```
Employee(employee_id, full_name, address, ssn_sin)
```

1.6 Role / Position
Employees may have various roles or positions

```
Role(role_id, role_name)
```

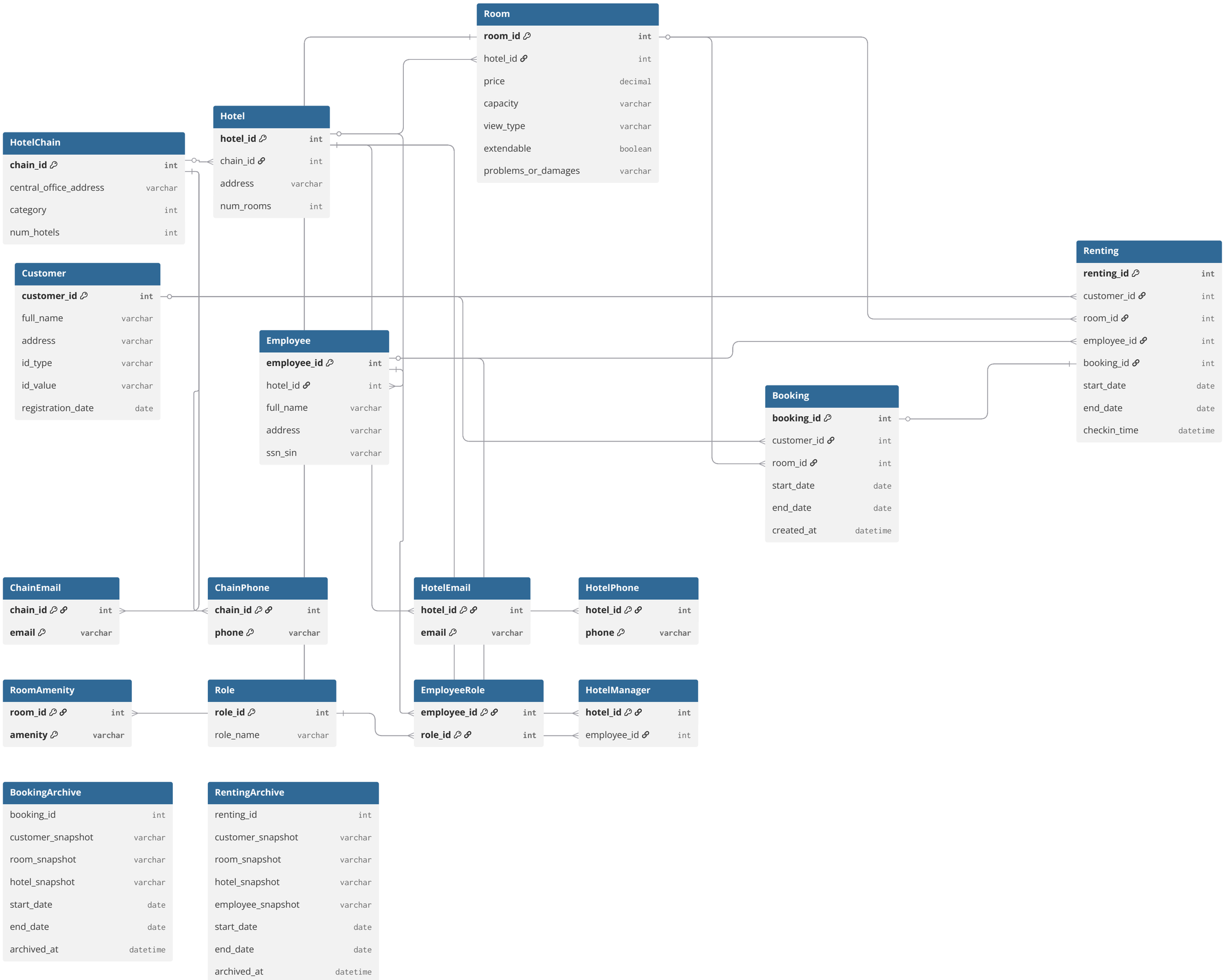
1.7 **Booking & Renting**
this two can switch, booking happens on a **Specific Data**

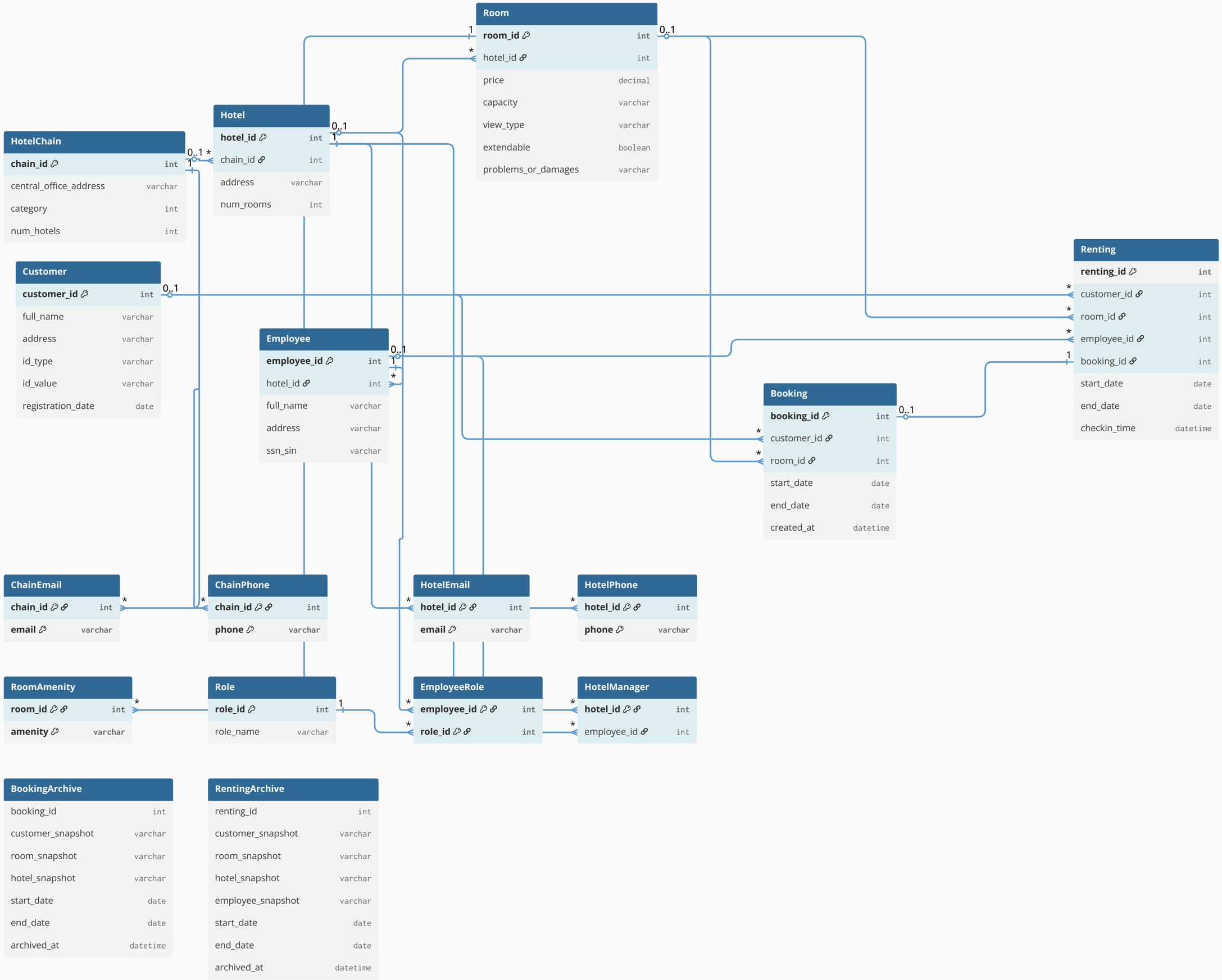
- Check-in: booking -> renting
- walk-in: no booking -> renting
    - employees do all this switches

```
// Booking
Booking(booking_id, start_date, end_date, booking_created_at)

// Renting
Renting(renting_id, start_date, end_date, checkin_time)
```

1.1 ER Diagram


Room
room_id int
hotel_id int
price decimal
capacity varchar
view_type varchar
extendable boolean
problems_or_damages varchar
Hotel
hotel_id int
chain_id int
address varchar
num_rooms int
HotelChain
chain_id int
central_office_address varchar
category int
num_hotels int
Customer
customer_id int
full_name varchar
address varchar
id_type varchar
id_value varchar
registration_date date
Renting
renting_id int
customer_id int
room_id int
employee_id int
booking_id int
start_date date
end_date date
checkin_time datetime
Employee
employee_id int
hotel_id int
full_name varchar
address varchar
ssn_sin varchar
Booking
booking_id int
customer_id int
room_id int
start_date date
end_date date
created_at datetime
ChainEmail
chain_id int
email varchar
ChainPhone
chain_id int
phone varchar
HotelEmail
hotel_id int
email varchar
HotelPhone
hotel_id int
phone varchar
RoomAmenity
room_id int
amenity varchar
Role
role_id int
role_name varchar
EmployeeRole
employee_id int
role_id int
HotelManager
hotel_id int
employee_id int
BookingArchive
booking_id int
customer_snapshot varchar
room_snapshot varchar
hotel_snapshot varchar
start_date date
end_date date
archived_at datetime
RentingArchive
renting_id int
customer_snapshot varchar
room_snapshot varchar
hotel_snapshot varchar
employee_snapshot varchar
start_date date
end_date date
archived_at datetime


Room
room_id int
hotel_id int
price decimal
capacity varchar
view_type varchar
extendable boolean
problems_or_damages varchar
Hotel
hotel_id int
chain_id int
address varchar
num_rooms int
HotelChain
chain_id int
central_office_address varchar
category int
num_hotels int
Customer
customer_id int
full_name varchar
address varchar
id_type varchar
id_value varchar
registration_date date
Employee
employee_id int
hotel_id int
full_name varchar
address varchar
ssn_sin varchar
Renting
renting_id int
customer_id int
room_id int
employee_id int
booking_id int
start_date date
end_date date
checkin_time datetime
Booking
booking_id int
customer_id int
room_id int
start_date date
end_date date
created_at datetime
ChainEmail
chain_id int
email varchar
ChainPhone
chain_id int
phone varchar
HotelEmail
hotel_id int
email varchar
HotelPhone
hotel_id int
phone varchar
RoomAmenity
room_id int
amenity varchar
Role
role_id int
role_name varchar
EmployeeRole
employee_id int
role_id int
HotelManager
hotel_id int
employee_id int
BookingArchive
booking_id int
customer_snapshot varchar
room_snapshot varchar
hotel_snapshot varchar
start_date date
end_date date
archived_at datetime
RentingArchive
renting_id int
customer_snapshot varchar
room_snapshot varchar
hotel_snapshot varchar
employee_snapshot varchar
start_date date
end_date date
archived_at datetime

b. Relational database schema

```
HOTEL_CHAIN(
    chain_id PK,
    central_office_address,
    category,
    num_hotels
)

CHAIN_EMAIL(
    chain_id PK FK → HOTEL_CHAIN.chain_id,
    email PK
)

CHAIN_PHONE(
    chain_id PK FK → HOTEL_CHAIN.chain_id,
    phone PK
)

HOTEL(
    hotel_id PK,
    chain_id FK → HOTEL_CHAIN.chain_id,
    address,
    num_rooms
)

HOTEL_EMAIL(
    hotel_id PK FK → HOTEL.hotel_id,
    email PK
)

HOTEL_PHONE(
    hotel_id PK FK → HOTEL.hotel_id,
    phone PK
)

ROOM(
    room_id PK,
    hotel_id FK → HOTEL.hotel_id,
    price,
    capacity,
    view_type,
    extendable,
    problems_or_damages
)

ROOM_AMENITY(
    room_id PK FK → ROOM.room_id,
    amenity PK
)

CUSTOMER(
    customer_id PK,
    full_name,
    address,
    id_type,
    id_value,
```

```
    registration_date
)

EMPLOYEE(
    employee_id PK,
    hotel_id FK → HOTEL.hotel_id,
    full_name,
    address,
    ssn_sin
)

ROLE(
    role_id PK,
    role_name
)

EMPLOYEE_ROLE(
    employee_id PK FK → EMPLOYEE.employee_id,
    role_id PK FK → ROLE.role_id
)

HOTEL_MANAGER(
    hotel_id PK FK → HOTEL.hotel_id,
    employee_id UNIQUE FK → EMPLOYEE.employee_id
)

BOOKING(
    booking_id PK,
    customer_id FK → CUSTOMER.customer_id,
    room_id FK → ROOM.room_id,
    start_date,
    end_date,
    created_at
)

RENTING(
    renting_id PK,
    customer_id FK → CUSTOMER.customer_id,
    room_id FK → ROOM.room_id,
    employee_id FK → EMPLOYEE.employee_id,
    booking_id UNIQUE NULL FK → BOOKING.booking_id,
    start_date,
    end_date,
    checkin_time
)

BOOKING_ARCHIVE(
    booking_id PK,
    customer_snapshot,
    room_snapshot,
    hotel_snapshot,
    start_date,
    end_date,
    archived_at
)

RENTING_ARCHIVE(
    renting_id PK,
```

```
    customer_snapshot,
    room_snapshot,
    hotel_snapshot,
    employee_snapshot,
    start_date,
    end_date,
    archived_at
)
```

c. Integrity Constraints

c.1 Primary Key Constraints

```
PK(HOTEL_CHAIN) = chain_id
PK(HOTEL) = hotel_id
PK(ROOM) = room_id
PK(CUSTOMER) = customer_id
PK(EMPLOYEE) = employee_id
PK(BOOKING) = booking_id
PK(RENTING) = renting_id
PK(EMPLOYEE_ROLE) = (employee_id, role_id)
PK(ROOM_AMENITY) = (room_id, amenity)
```

Explanation: primary keys ensure entity integrity by guaranteeing uniqueness and NOT-NULL id of each tuple

c.2 Referential Integrity Constraints

```
FK(HOTEL.chain_id) → HOTEL_CHAIN.chain_id
FK(ROOM.hotel_id) → HOTEL.hotel_id
FK(EMPLOYEE.hotel_id) → HOTEL.hotel_id
FK(BOOKING.customer_id) → CUSTOMER.customer_id
FK(BOOKING.room_id) → ROOM.room_id
FK(RENTING.customer_id) → CUSTOMER.customer_id
FK(RENTING.room_id) → ROOM.room_id
FK(RENTING.employee_id) → EMPLOYEE.employee_id
FK(EMPLOYEE_ROLE.employee_id) → EMPLOYEE.employee_id
FK(EMPLOYEE_ROLE.role_id) → ROLE.role_id
```

Explanation: ensure referential integrity by preventing references to NOT-EXISTING entities

c.3 Domain & Attribute Constraints

```
HOTEL_CHAIN.category ∈ {1,2,3,4,5}

ROOM.price > 0

BOOKING.start_date < BOOKING.end_date

RENTING.start_date < RENTING.end_date

CUSTOMER.id_type ∈ {SSN, SIN, DRIVER_LICENSE}

ROOM.extendable ∈ {TRUE, FALSE}
```

Explanation: DOmain constraints restrict attribute values to valid ranges consistent with the business semantics of the system.

c.4 User-Defined Constraints

1. every hotel must have exactly one manager

```
For every hotel h ∈ HOTEL,
there exists exactly one employee e ∈ EMPLOYEE
such that (h, e) ∈ HOTEL_MANAGER.
```

2. no double booking of the same room at the same time

```
For any two bookings b1, b2:
if b1.room_id = b2.room_id
and their date intervals overlap,
then b1.booking_id = b2.booking_id.
```

Prevents overlapping bookings for the same room.

3. A booking can be transformed into at most one renting

```
For any renting r:
    if r.booking_id is not NULL,
    then there exists exactly one booking b such that r.booking_id = b.booking_id.
```

Ensures one-to-one transformation between booking and renting.